

ФН
АЛГОРИТМИЗАЦИЯ И ПРОГРАММИРОВАНИЕ
Модуль 1

Лабораторная работа №1
«IDLE. Ввод и вывод данных. Математические операции»

Цель работы:

Познакомиться со средой разработки IDLE, научиться применять команды ввода и вывода данных, получить навыки работы с математическими операциями над числовыми данными в Python.

Теоретический материал:

1. Среда разработки IDLE

IDLE – интегрированная среда разработки, включающая интерактивную оболочку для выполнения отдельных фрагментов кода Python и текстовый редактор с расширенным набором возможностей по написанию законченных программ на Python (подсветка синтаксиса, просмотр объектов, отладка и т.п.). IDLE расшифровывается как «Integrated Development and Learning Environment», что в переводе означает «Интегрированная среда разработки и изучения». Из названия прямо следует, что IDLE предназначена не только для написания программ, но и для изучения языка Python. Сама IDLE поставляется в комплекте с интерпретатором Python и отдельной установки не требует.

Интерпретатор последней версии Python можно скачать бесплатно с сайта www.python.org. Зайдя на сайт, выбираем пункт меню «Downloads» и свою операционную систему (Windows или macOS). Далее нажимаем на ссылку «Latest Python 3 Release» и прокручиваем открывшуюся страницу вниз пока не увидим таблицу со ссылками на интерпретатор для различных операционных систем. Выбираем «Windows installer (64-bit)» для Windows или «macOS installer» для macOS. В результате на компьютер будет загружен установочный файл. Запускаем его и следуем появляющимся инструкциям по установке.

В Linux в зависимости от используемого дистрибутива интерпретатор Python либо уже предустановлен, либо может быть установлен с помощью менеджера пакетов.

После установки для запуска IDLE в Windows необходимо перейти в папку Python в меню «Пуск» и найти там ярлык с именем «IDLE (Python 3.X XX-bit)». Будет запущена интерактивная оболочка. Чтобы открыть текстовый редактор, необходимо в главном меню этой оболочки выбрать «File» и далее «New File». По состоянию на август 2026 года последней версией Python является 3.14.7.

2. Команды ввода и вывода данных

Вывести на экран результаты работы программы можно с помощью функции `print`. В наиболее общем виде функция имеет следующий формат:

```
print(objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

В этой функции

`objects` – один или несколько объектов, которые нужно вывести;

`sep` – разделитель между объектами (по умолчанию пробел);

`end` – символ завершения строки (по умолчанию перевод строки);

`file` – объект файла, в который направлен вывод (по умолчанию на экран);

`flush` – принудительный сброс буфера вывода (по умолчанию сброса нет).

Любой из этих аргументов при вызове функции можно опустить. Тогда для отсутствующих аргументов будет использоваться значение по умолчанию.

Аргумент `objects` в функции – это то, что мы хотим вывести на экран. Выводить можно строки, числа, переменные, выражения и т.д. После вывода `objects` функция `print` по умолчанию добавляет символ перевода строки `\n`. Он на экране не отображается, но его присутствие указывает интерпретатору, что в этом месте надо перейти на новую строку. Например, две команды

```
print('Строка 1')  
print('Строка 2')
```

выведут на экран две строки

```
Строка 1  
Строка 2
```

Если необходимо вывести результат на той же строке, то в функции `print` данные указываются через запятую:

```
print('Строка1', 'Строка2')
```

Результат:

```
Строка1 Строка2
```

Между выводимыми строками по умолчанию вставляется пробел. С помощью параметра `sep` можно указать другой символ или последовательность символов. Например, запятую и пробел:

```
print('Строка1', 'Строка2', sep=', ')
```

Результат:

```
Строка1, Строка2
```

Как было сказано ранее, после вывода объектов в конце добавляется символ перевода строки. Если необходимо произвести вывод при следующем вызове функции `print` на той же строке, то в параметре `end` следует указать другой символ или другую последовательность символов. Например, запятую и пробел:

```
print('Строка1', 'Строка2', sep=', ', end=', ')  
print('Строка3')
```

Результат:

Строка1, Строка2, Строка3

Для вывода содержимого переменной достаточно указать ее имя в качестве аргумента функции `print`:

```
name='Алена'  
print('Меня зовут', name)
```

Результат

Меня зовут Алена

Пример вывода числа и выражения:

```
print(3, 4+5)
```

Результат

3 9

Для ввода данных предназначена функция `input`. Она имеет следующий формат:

```
переменная=input(сообщение)
```

Интерпретатор выводит на экран сообщение и переходит в режим ожидания ввода. После ввода пользователем данных и нажатия клавиши `Enter` интерпретатор записывает результат ввода в переменную. Переменная и сообщение при вызове функции могут быть опущены.

Пример программы:

```
name=input('Введите ваше имя:')  
print('Привет,',name, '!')  
input('Нажмите <Enter> для завершения...')
```

Результат:

Введите ваше имя:Иван
Привет, Иван !
Нажмите <Enter> для завершения...

Функция `input` всегда возвращает данные в виде строки, даже если пользователь вводит число. Для получения числовых значений необходимо использовать функции преобразования типов данных:

`int(значение)` – преобразует значение в целое число,

`float(значение)` – преобразует значение в вещественное число.

Например, следующая программа запрашивает у пользователя произвольное целое число, увеличивает его на 2 и выводит на экран:

```
number=int(input('Введите целое число: '))  
print(number+2)
```

Результат:

Введите целое число: 3
5

В данной программе делается явное преобразование типов. Функция `input` возвращает введенное пользователем целое число в виде строки, а функция `int` приводит эту строку к целочисленному типу.

В примере пользователь вводит число 3. Функция `input` возвращает строку '3' и передает ее в качестве входного аргумента функции `int`. Далее функция `int` переводит строку '3' в число 3, после чего это число присваивается переменной `number`.

Если в данной программе явно не сделать преобразование типов

```
int(input(...)),
```

то в переменную `number` запишется строка и при попытке выполнения операции сложения `number+2` программа остановится и выдаст ошибку, поскольку операция сложения строки и числа в Python неопределенна.

3. Форматированный вывод данных

Форматирование строк – это процесс создания новых строк по заданному шаблону, путем подстановки в него значений переменных или выражений. Форматировать строки можно с помощью оператора `%`. Он задает место в строке, куда должно быть подставлено значение переменной, и определяет, как будет выглядеть выводимое значение на экране в зависимости от типа переменной. Например, выражение `%s` указывает, что в этом месте надо вставить строку, `%d` означает целое число, а `%f` – число с плавающей запятой. Такие выражения называются спецификаторами формата, поскольку они определяют, как именно нужно интерпретировать и отображать значение переменной при выводе на экран.

Приведем еще пару примеров. Спецификатор `%.2f` говорит, что число с плавающей запятой при выводе на экран необходимо округлить до двух знаков после запятой. Спецификатор `%5d` указывает, что минимальное количество цифр в выводимом целом числе должно быть 5. Если цифр по факту меньше, то недостающие цифры при выводе заменяются пробелами слева от числа.

Оператор `%` считается классическим способом форматирования строк, но в современном Python уступает место `f`-строкам. Приведем примеры использования оператора `%`:

А. Вывод значения одной переменной

```
name = 'Алексей'
print('Привет, %s!' % name)
```

Результат:

```
Привет, Алексей!
```

Замечание:

Обратите внимание, что переменная `name` пишется сразу после формируемой строки и предваряется символом `%`, при этом конструкция `%name` отделяется от строки только пробелом.

Б. Вывод значений нескольких переменных

```
age = 20
```

```
gra = 4.75
print("Студент %s, возраст: %d, средний балл: %.1f" %(name, age, gra))
```

Результат:

Студент Алексей, возраст: 20, средний балл: 4.8

Замечание:

Спецификатор `%.1f` в примере округляет выводимое число с плавающей точкой до одного знака после запятой – переменная `gra` содержит значение 4.75, которое выводится как 4.8.

Все переменные `name`, `age` и `gra` перечисляются после самой формируемой строки в порядке вставки их значений, т.е. первый спецификатор `%s` соответствует первой переменной `name` и т.д. Переменные обязательно должны быть выделены круглыми скобками и предваряться символом `%`.

Порядок выполнения работы:

1. Установить интерпретатор языка Python и среду разработки IDLE на свой рабочий компьютер.
2. Ознакомиться с предложенными к решению задачами (см. ниже).
3. Разработать алгоритмы решения задач. Для каждой задачи подобрать наиболее компактный алгоритм решения.
4. Написать программы, реализующие разработанные алгоритмы с помощью перечисленных в разделе «Теоретический материал» конструкций языка Python. Отладить написанные программы.
5. Подготовить отчет.

Структура отчета:

1. Титульный лист.
2. Цель работы.
3. Задачи с решением. Для каждой задачи необходимо привести:
 - Текст задачи.
 - Листинг программы на языке Python с подробными комментариями к каждому блоку кода – что этот блок кода делает.
 - Рисунок с изображением окна интерактивной оболочки, демонстрирующего результаты работы программы.
4. Заключение.

Задачи:

Задача 1

Написать программу вывода сообщения «Привет, мир!»

Задача 2

Написать программу, которая запрашивает у пользователя его имя и выводит на экран сообщение вида «Привет, ИМЯ», где ИМЯ – введенное пользователем имя.

Задача 3

Написать программу расчета по заданным формулам.

Исходные данные (вводятся пользователем в консольном режиме с клавиатуры): значения переменных a , b , c вещественного типа.

Формулы:

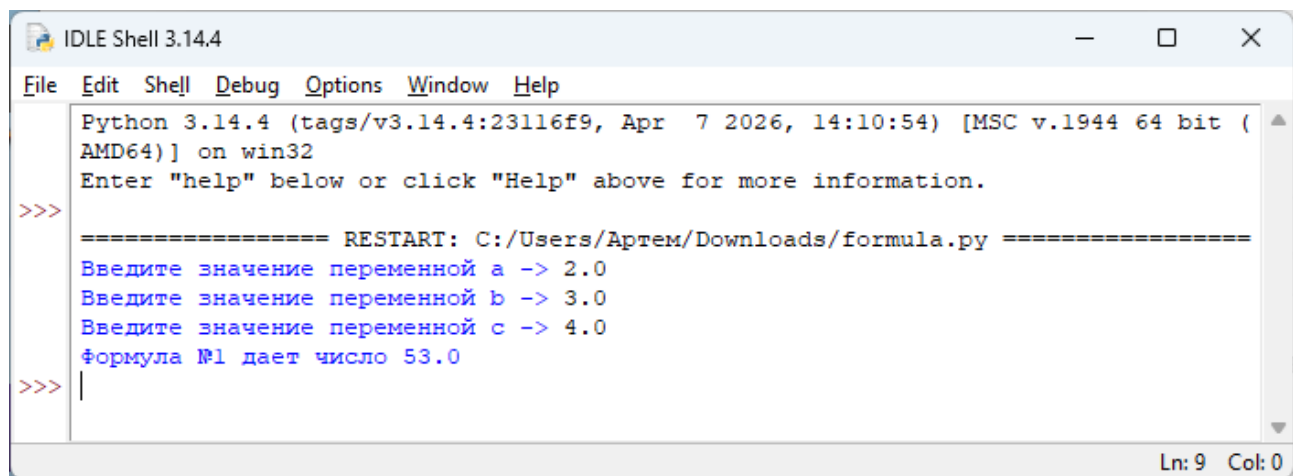
1. $6a^3 - \frac{8b^2}{3c} + 11.$

2. $6a^2 + 2\frac{a}{b} + b^5.$

3. $2abc^5$

Проверить правильность программной реализации каждой формулы с помощью калькулятора.

Рекомендуемый вид экрана программы для первой формулы приведен на рис. 1.



```
IDLE Shell 3.14.4
File Edit Shell Debug Options Window Help
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Aptem/Downloads/formula.py =====
Введите значение переменной a -> 2.0
Введите значение переменной b -> 3.0
Введите значение переменной c -> 4.0
Формула №1 дает число 53.0
>>>
```

Рис. 1. Рекомендуемый вид экрана программы для первой формулы задачи 3

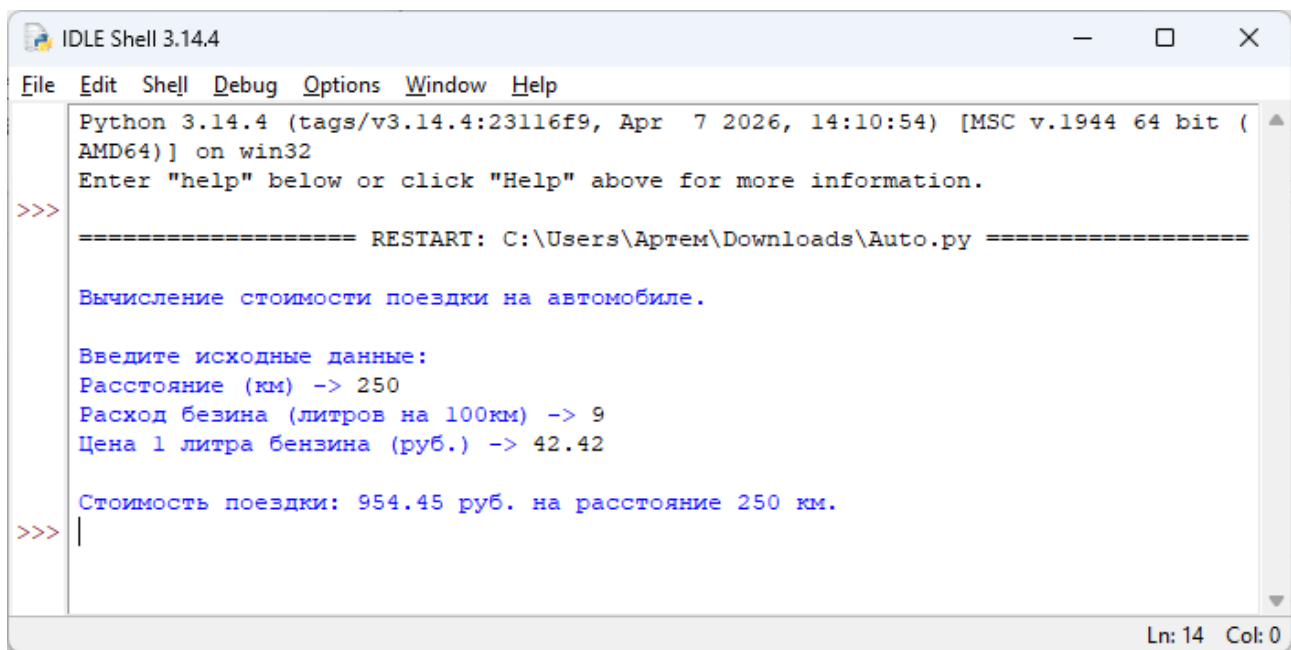
Задача 4

Написать программу вычисления стоимости поездки на автомобиле.

Исходные данные (вводятся пользователем в консольном режиме с клавиатуры):

- расстояние (км);
- количество бензина (литров), потребляемое автомобилем на 100км пробега;
- цена одного литра бензина (руб.)

Рекомендуемый вид экрана программы приведен на рис. 2.



```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\Арте\Downloads\Auto.py =====

Вычисление стоимости поездки на автомобиле.

Введите исходные данные:
Расстояние (км) -> 250
Расход бензина (литров на 100км) -> 9
Цена 1 литра бензина (руб.) -> 42.42

Стоимость поездки: 954.45 руб. на расстояние 250 км.

>>> |
```

Рис. 2. Рекомендуемый вид экрана программы задачи 4

Задача 5

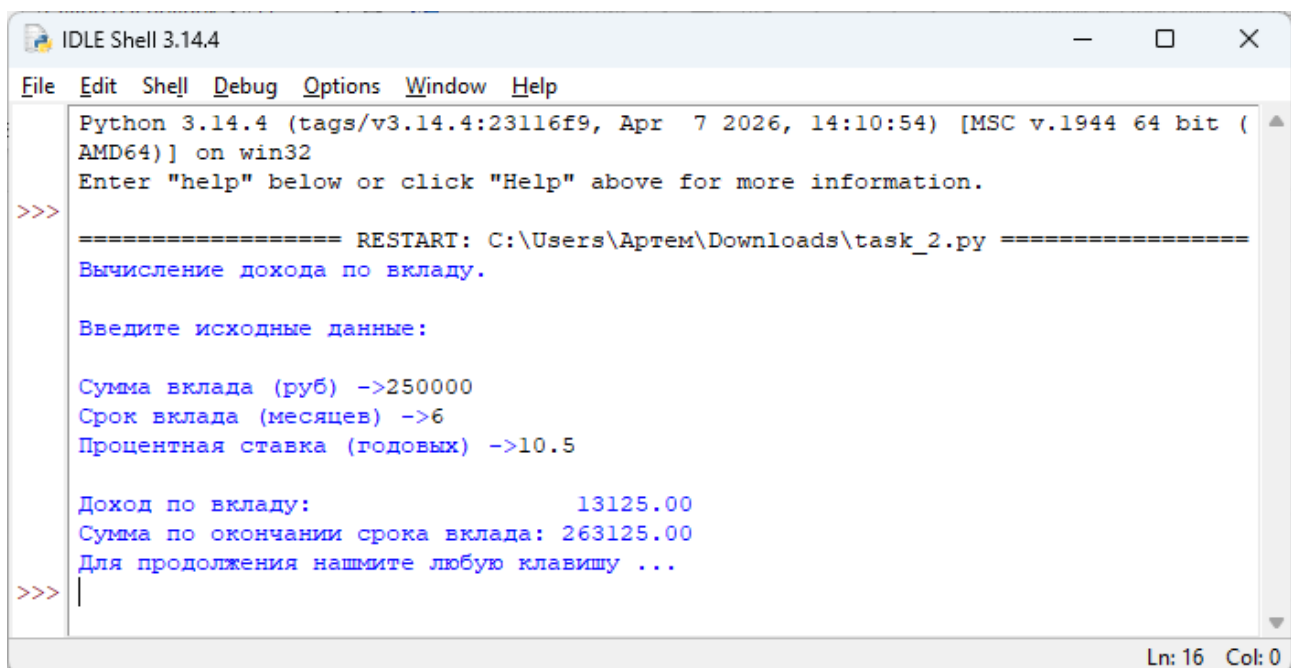
Написать программу вычисления дохода по вкладу.

Исходные данные (вводятся пользователем в консольном режиме с клавиатуры):

- сумма вклада (руб.);
- срок вклада (месяцев);
- процентная ставка (% годовых).

Рекомендуемый вид экрана программы приведен на рис. 3.

Рекомендуемый вид экрана программы:



```
Python 3.14.4 (tags/v3.14.4:23116f9, Apr 7 2026, 14:10:54) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:\Users\Арте\Downloads\task_2.py =====

Вычисление дохода по вкладу.

Введите исходные данные:

Сумма вклада (руб) ->250000
Срок вклада (месяцев) ->6
Процентная ставка (годовых) ->10.5

Доход по вкладу: 13125.00
Сумма по окончании срока вклада: 263125.00
Для продолжения нажмите любую клавишу ...

>>> |
```

Рис. 3. Рекомендуемый вид экрана программы задачи 5

Задача 6

Написать программу, которая для введённого с клавиатуры числа (например, 2026) выводит строку следующего вида «Вы ввели число 2026, в нём 4 цифры, последняя из которых равна 6».

Контрольные вопросы:

1. Для чего в print нужны параметры sep и end? Чем они отличаются?

2. Что выведет фрагмент

```
print('Строка1', 'Строка2', sep=', ', end=', ')\nprint('Строка3')
```

и почему результат окажется в одной строке?

3. Что возвращает функция input? Зависит ли тип результата от того, ввёл пользователь число или текст?

4. Зачем вокруг input пишут int(...) или float(...)? Когда какую функцию выбирать?

5. Почему программа

```
number = input('Введите целое число: ')\nprint(number + 2)
```

завершится ошибкой?

6. Какими операторами в Python записываются сложение, вычитание, умножение, деление и возведение в степень?

7. Что делает оператор %? Как с его помощью получить последнюю цифру целого числа?

8. Как на Python записать формулу $6a^3 - \frac{8b^2}{3c} + 11$? Какие скобки здесь

обязательны, а какие можно опустить?

9. Что произойдёт, если в задаче про стоимость поездки не преобразовать ввод в float (или int) и попытаться умножить «расстояние» на «цену литра»?

10. Операция ** это возведение в степень. Каков приоритет операций **, *, /, +, -? Что будет вычислено раньше в выражении 6 * a ** 3? Какой результат выполнения 2**3?